

INFO601 – Assessment 2

Advanced SQL

SQL Stored Procedures, Views and Triggers

Student Name	Tirth Patel
Student ID	
Module	Data Modelling & SQL
Assessment	Assessment 2

Assessment Evidence – Page 2

Create Client Table & Insert Data

SQL Script Screenshot

```
CREATE TABLE dbo.Client (  
    ClientID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    Name NVARCHAR(100) NOT NULL,  
    ContactEmail NVARCHAR(255) NOT NULL UNIQUE,  
    ContactPhone NVARCHAR(20) NOT NULL,  
    Address NVARCHAR(255) NOT NULL  
);  
GO  
  
CREATE TABLE dbo.Account (
```

Result Screenshot

Results		Messages			
	ClientID	Name	ContactEmail	ContactPhone	Address
1	1	Tech Corp	contact@techcorp.com	555-1234	123 Tech Lane
2	2	BlueWave Ltd	admin@bluewave.com	555-5678	45 Ocean Road
3	3	SecureBank	it@securebank.com	555-9012	77 Finance Ave
4	4	GreenHealth	ops@greenhealth.com	555-3456	12 Wellness St
5	5	EduNext	support@edunext.com	555-7890	88 Campus Dr

Create Account Table & Insert Data

SQL Script Screenshot

```
CREATE TABLE dbo.Account (
    AccountID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    AccountName NVARCHAR(100) NOT NULL,
    ClientID INT NOT NULL,
    SubscriptionDate DATE NOT NULL,
    AccountStatus NVARCHAR(20) NOT NULL CONSTRAINT DF_AccountStatus DEFAULT ('Active'),
    LastModifiedDate DATETIME2(0) NOT NULL CONSTRAINT DF_AccountLastModifiedDate DEFAULT (SYSDATETIME()),
    CONSTRAINT FK_Account_Client
        FOREIGN KEY (ClientID) REFERENCES dbo.Client(ClientID),
    CONSTRAINT CK_AccountStatus
        CHECK (AccountStatus IN ('Active', 'Suspended', 'Closed'))
);
GO
```

Result Screenshot

	AccountID	AccountName	ClientID	SubscriptionDate	AccountStatus	LastModifiedDate
1	1	Main Account	1	2024-01-15	Active	2026-05-16 15:20:16
2	2	Backup Account	2	2024-01-20	Active	2026-05-16 15:20:16
3	3	Finance Account	3	2024-01-25	Suspended	2026-05-16 15:20:16
4	4	Operations Account	4	2024-01-30	Active	2026-05-16 15:20:16
5	5	Training Account	5	2024-02-01	Closed	2026-05-16 15:20:16
	ServiceID	ServiceName	ServiceDescription			

Assessment Evidence – Page 4

Create Services Table & Insert Data

SQL Script Screenshot

```
29  
30 CREATE TABLE dbo.Services (  
31     ServiceID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
32     ServiceName NVARCHAR(100) NOT NULL UNIQUE,  
33     ServiceDescription NVARCHAR(255) NOT NULL  
34 );  
35 GO
```

Result Screenshot

	ServiceID	ServiceName	ServiceDescription
1	1	Cloud Storage	Secure cloud storage with encrypted backups
2	2	Virtual Machines	Scalable virtual machine hosting
3	3	Cybersecurity Monitoring	24/7 monitoring of security events and threats

Create Subscriptions Table & Insert Data

SQL Script Screenshot

```
CREATE TABLE dbo.Subscriptions (  
    SubscriptionID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    AccountID INT NOT NULL,  
    ServiceID INT NOT NULL,  
    SubscriptionDate DATE NOT NULL,  
    SubscriptionStatus NVARCHAR(20) NOT NULL CONSTRAINT DF_SubscriptionStatus DEFAULT ('Active'),  
    CONSTRAINT FK_Subscriptions_Account  
        FOREIGN KEY (AccountID) REFERENCES dbo.Account(AccountID),  
    CONSTRAINT FK_Subscriptions_Service  
        FOREIGN KEY (ServiceID) REFERENCES dbo.Services(ServiceID),  
    CONSTRAINT UQ_Subscriptions_Account_Service_Date  
        UNIQUE (AccountID, ServiceID, SubscriptionDate),  
    CONSTRAINT CK_SubscriptionStatus  
        CHECK (SubscriptionStatus IN ('Active', 'Suspended', 'Cancelled'))  
);  
GO
```

Result Screenshot

	SubscriptionID	AccountID	ServiceID	SubscriptionDate	SubscriptionStatus
1	1	1	1	2024-02-15	Active
2	2	1	3	2024-03-01	Active
3	3	2	2	2024-02-20	Active
4	4	3	1	2024-03-05	Suspended
5	5	4	3	2024-03-10	Active

Assessment Evidence – Page 6

Create Billing Table & Insert Data

SQL Script Screenshot

```
CREATE TABLE dbo.Billing (  
    BillingID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,  
    SubscriptionID INT NOT NULL,  
    BillingDate DATE NOT NULL,  
    Amount DECIMAL(10,2) NOT NULL,  
    PaymentStatus NVARCHAR(20) NOT NULL,  
    CONSTRAINT FK_Billing_Subscription  
        FOREIGN KEY (SubscriptionID) REFERENCES dbo.Subscriptions(SubscriptionID),  
    CONSTRAINT CK_Billing_Amount  
        CHECK (Amount > 0),  
    CONSTRAINT CK_PaymentStatus  
        CHECK (PaymentStatus IN ('Paid', 'Unpaid', 'Pending'))  
);  
GO
```

Result Screenshot

	BillingID	SubscriptionID	BillingDate	Amount	PaymentStatus
1	1	1	2024-02-28	100.00	Paid
2	2	2	2024-03-31	150.00	Unpaid
3	3	3	2024-02-28	200.00	Paid
4	4	4	2024-03-31	120.00	Pending
5	5	5	2024-04-30	175.00	Paid

Assessment Evidence – Page 7

Create CybersecurityIncidents Table & Insert Data

SQL Script Screenshot

```
CREATE TABLE dbo.CybersecurityIncidents (
    IncidentID INT IDENTITY(1,1) NOT NULL PRIMARY KEY,
    AccountID INT NOT NULL,
    IncidentDate DATE NOT NULL,
    IncidentType NVARCHAR(100) NOT NULL,
    IncidentStatus NVARCHAR(20) NOT NULL,
    SeverityLevel NVARCHAR(20) NOT NULL,
    Description NVARCHAR(500) NOT NULL,
    CONSTRAINT FK_Incidents_Account
        FOREIGN KEY (AccountID) REFERENCES dbo.Account(AccountID),
    CONSTRAINT CK_IncidentStatus
        CHECK (IncidentStatus IN ('Open', 'Investigating', 'Resolved')),
    CONSTRAINT CK_SeverityLevel
        CHECK (SeverityLevel IN ('Low', 'Medium', 'High', 'Critical'))
);
GO
```

Result Screenshot

	IncidentID	AccountID	IncidentDate	IncidentType	IncidentStatus	SeverityLevel	Description
1	1	1	2024-02-20	DDoS Attack	Investigating	High	Traffic spike detected and blocked
2	2	2	2024-02-25	Phishing Attempt	Resolved	Medium	Suspicious email quarantined
3	3	3	2024-03-02	Malware Detec.	Open	Critical	Endpoint isolated for investigation
4	4	4	2024-03-15	Unauthorized L.	Resolved	High	Login attempts traced and block.
5	5	1	2024-04-01	Port Scan	Open	Low	Repeated scan from external IP

Query executed successfully.

Assessment Evidence – Page 8

View: Clients and their Accounts

SQL Script Screenshot

```
5  VIEW 1: Clients and their Accounts
6  ===== */
7  CREATE OR ALTER VIEW dbo.vw_ClientsAccounts
8  AS
9  SELECT
10     c.ClientID,
11     c.Name AS ClientName,
12     c.ContactEmail,
13     a.AccountID,
14     a.AccountName,
15     a.AccountStatus
16 FROM dbo.Client c
17 LEFT JOIN dbo.Account a
18     ON c.ClientID = a.ClientID;
19 GO
20
```

Result Screenshot

	ClientID	ClientName	ContactEmail	AccountID	AccountName	AccountStatus
1	1	Tech Corp	contact@techcorp.com	1	Main Account	Active
2	2	BlueWave Ltd	admin@bluewave.com	2	Backup Account	Active
3	3	SecureBank	it@securebank.com	3	Finance Account	Suspended
4	4	GreenHealth	ops@greenhealth.com	4	Operations Account	Active
5	5	EduNext	support@edunext.com	5	Training Account	Closed

View: Accounts and Services

SQL Script Screenshot

```
22 VIEW 2: Accounts and subscribed Services
23 ===== */
24 CREATE OR ALTER VIEW dbo.vw_AccountsServices
25 AS
26 SELECT
27     a.AccountID,
28     a.AccountName,
29     s.ServiceID,
30     s.ServiceName,
31     sub.SubscriptionDate,
32     sub.SubscriptionStatus
33 FROM dbo.Subscriptions sub
34 INNER JOIN dbo.Account a
35     ON sub.AccountID = a.AccountID
36 INNER JOIN dbo.Services s
37     ON sub.ServiceID = s.ServiceID;
38 GO
39
```

Result Screenshot

	AccountID	AccountName	ServiceID	ServiceName	SubscriptionDate	SubscriptionStatus
1	1	Main Account	1	Cloud Storage	2024-02-15	Active
2	1	Main Account	3	Cybersecurity Monitoring	2024-03-01	Active
3	2	Backup Account	2	Virtual Machines	2024-02-20	Active
4	3	Finance Account	1	Cloud Storage	2024-03-05	Suspended
5	4	Operations Account	3	Cybersecurity Monitoring	2024-03-10	Active

Assessment Evidence – Page 10

View: Billing Information

SQL Script Screenshot

```
VIEW 3: Billing Information
===== */
CREATE OR ALTER VIEW dbo.vw_BillingInformation
AS
SELECT
    b.BillingID,
    b.BillingDate,
    b.Amount,
    b.PaymentStatus,
    a.AccountName,
    s.ServiceName
FROM dbo.Billing b
INNER JOIN dbo.Subscriptions sub
    ON b.SubscriptionID = sub.SubscriptionID
INNER JOIN dbo.Account a
    ON sub.AccountID = a.AccountID
INNER JOIN dbo.Services s
    ON sub.ServiceID = s.ServiceID;
GO
```

Result Screenshot

	BillingID	BillingDate	Amount	PaymentStatus	AccountName	ServiceName
1	1	2024-02-28	100.00	Paid	Main Account	Cloud Storage
2	2	2024-03-31	150.00	Unpaid	Main Account	Cybersecurity Monitoring
3	3	2024-02-28	200.00	Paid	Backup Account	Virtual Machines
4	4	2024-03-31	120.00	Pending	Finance Acco...	Cloud Storage
5	5	2024-04-30	175.00	Paid	Operations Ac...	Cybersecurity Monitoring

View: Cybersecurity Incidents

SQL Script Screenshot

```
61  /* =====
62  VIEW 4: Cybersecurity Incidents
63  ===== */
64  CREATE OR ALTER VIEW dbo.vw_CybersecurityIncidents
65  AS
66  SELECT
67      i.IncidentID,
68      i.IncidentDate,
69      i.IncidentType,
70      i.IncidentStatus,
71      i.SeverityLevel,
72      a.AccountName
73  FROM  dbo.CybersecurityIncidents i
74  INNER JOIN  dbo.Account a
75      ON i.AccountID = a.AccountID;
76  GO
```

Result Screenshot

	IncidentID	IncidentDate	IncidentType	IncidentStatus	SeverityLevel	AccountName
1	1	2024-02-20	DDoS Attack	Investigating	High	Main Account
2	2	2024-02-25	Phishing Attempt	Resolved	Medium	Backup Account
3	3	2024-03-02	Malware Detec...	Open	Critical	Finance Acco...
4	4	2024-03-15	Unauthorized L...	Resolved	High	Operations Ac...
5	5	2024-04-01	Port Scan	Open	Low	Main Account

✔ Query executed successfully.

Assessment Evidence – Page 12

Stored Procedure: Insert Client

SQL Script Screenshot

```
79  =====
80  PROCEDURE 1: Insert Client
81  ===== */
82  CREATE OR ALTER PROCEDURE dbo.usp_InsertClient
83      @Name NVARCHAR(100),
84      @ContactEmail NVARCHAR(255),
85      @ContactPhone NVARCHAR(20),
86      @Address NVARCHAR(255)
87  AS
88  BEGIN
89      INSERT INTO dbo.Client
90      (Name, ContactEmail, ContactPhone, Address)
91      VALUES
92      (@Name, @ContactEmail, @ContactPhone, @Address);
93  END;
94  GO
```

Result Screenshot

Results		Messages			
	ClientID	Name	ContactEmail	ContactPhone	Address
1	1	Tech Corp	contact@techcorp.com	555-1234	123 Tech Lane
2	2	BlueWave Ltd	admin@bluewave.com	555-5678	45 Ocean Road
3	3	SecureBank	it@securebank.com	555-9012	77 Finance Ave
4	4	GreenHealth	ops@greenhealth.com	555-3456	12 Wellness St
5	5	EduNext	support@edunext.com	555-7890	88 Campus Dr
6	6	Test Company	test@test.com	555-9999	Test Address

Stored Procedure: Update Account

SQL Script Screenshot

```

96  PROCEDURE 2: Update Account
97  ===== */
98  CREATE OR ALTER PROCEDURE dbo.usp_UpdateAccount
99      @AccountID INT,
100      @AccountName NVARCHAR(100),
101      @AccountStatus NVARCHAR(20)
102  AS
103  BEGIN
104      UPDATE dbo.Account
105      SET
106          AccountName = @AccountName,
107          AccountStatus = @AccountStatus
108      WHERE AccountID = @AccountID;
109  END;
110  GO
111
112  /* =====
113  PROCEDURE 3: Delete Cybersecurity Incident

```

Result Screenshot

Results		Messages				
	AccountID	AccountName	ClientID	SubscriptionDate	AccountStatus	LastModifiedDate
1	1	Updated Main Account	1	2024-01-15	Active	2026-05-16 17:51:05
2	2	Backup Account	2	2024-01-20	Active	2026-05-16 15:20:16
3	3	Finance Account	3	2024-01-25	Suspended	2026-05-16 15:20:16
4	4	Operations Account	4	2024-01-30	Active	2026-05-16 15:20:16
5	5	Training Account	5	2024-02-01	Closed	2026-05-16 15:20:16

Stored Procedure: Delete Cybersecurity Incident

SQL Script Screenshot

```
=====
PROCEDURE 3: Delete Cybersecurity Incident
===== */
CREATE OR ALTER PROCEDURE dbo.usp_DeleteCybersecurityIncident
    @IncidentID INT
AS
BEGIN
    DELETE FROM dbo.CybersecurityIncidents
    WHERE IncidentID = @IncidentID;
END;
GO
```

Result Screenshot

	IncidentID	AccountID	IncidentDate	IncidentType	IncidentStatus	SeverityLevel	Description
1	2	2	2024-02-25	Phishing Attempt	Resolved	Medium	Suspicious email quarantined
2	3	3	2024-03-02	Malware Detected	Open	Critical	Endpoint isolated for investigation
3	4	4	2024-03-15	Unauthorized Login	Resolved	High	Login attempts traced and blocked
4	5	1	2024-04-01	Port Scan	Open	Low	Repeated scan from external IP

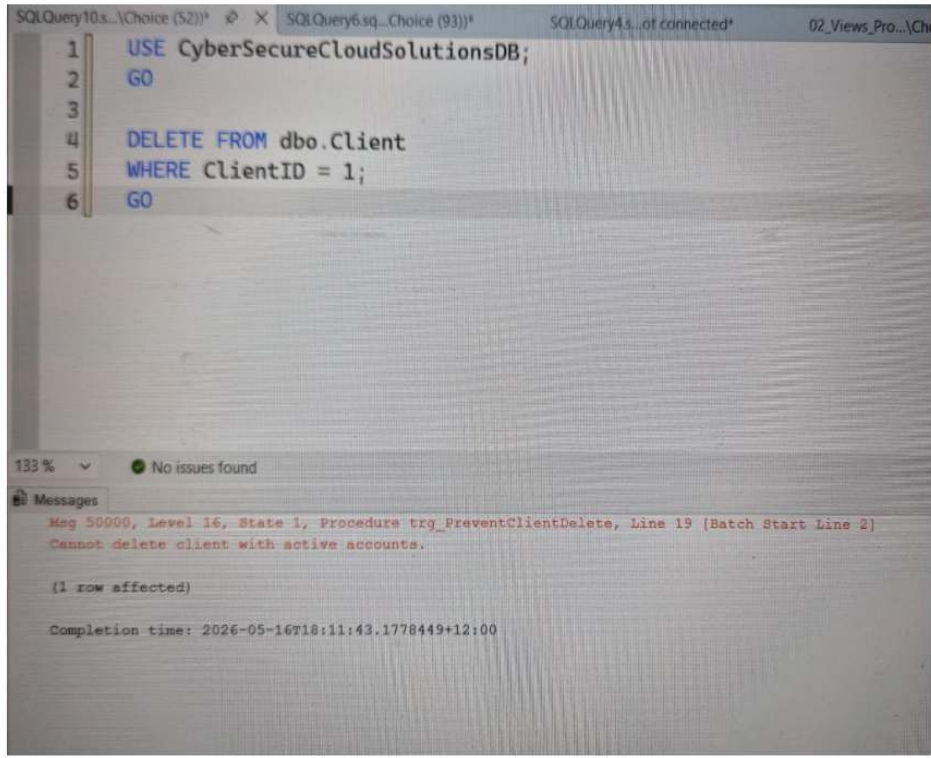
Trigger: Prevent Delete Client with Active Accounts

SQL Script Screenshot

```
139  /* =====
140  TRIGGER 2:
141  Prevent deleting client with active accounts
142  ===== */
143  CREATE OR ALTER TRIGGER dbo.trg_PreventClientDelete
144  ON dbo.Client
145  INSTEAD OF DELETE
146  AS
147  BEGIN
148      IF EXISTS (
149          SELECT 1
150          FROM dbo.Account a
151          INNER JOIN deleted d
152              ON a.ClientID = d.ClientID
153          WHERE a.AccountStatus = 'Active'
154      )
155      BEGIN
156          RAISERROR ('Cannot delete client with active accounts.', 16, 1);
157          RETURN;
158      END
159
160      DELETE FROM dbo.Client
161      WHERE ClientID IN (SELECT ClientID FROM deleted);
162  END;
163  GO
```

Result Screenshot

Assessment Evidence – Page 16



The screenshot displays the SQL Server Enterprise Manager interface. The top pane shows a query window with the following SQL code:

```
1 USE CyberSecureCloudSolutionsDB;  
2 GO  
3  
4 DELETE FROM dbo.Client  
5 WHERE ClientID = 1;  
6 GO
```

The bottom pane shows the Messages window with the following output:

```
Msg 50000, Level 16, State 1, Procedure trg_PreventClientDelete, Line 19 [Batch Start Line 2]  
Cannot delete client with active accounts.  
  
(1 row affected)  
  
Completion time: 2026-05-16T18:11:43.1778449+12:00
```

Assessment Evidence – Page 17

Trigger: Update LastModifiedDate

SQL Script Screenshot

```
126 update LastModifiedDate automatically
127 ===== */
128 CREATE OR ALTER TRIGGER dbo.trg_UpdateLastModifiedDate
129 ON dbo.Account
130 AFTER UPDATE
131 AS
132 BEGIN
133     UPDATE dbo.Account
134     SET LastModifiedDate = GETDATE()
135     WHERE AccountID IN (SELECT AccountID FROM inserted);
136 END;
137 GO
138
```

Result Screenshot

Results			
Messages			
AccountID	AccountName	LastModifiedDate	
1	Updated Main Account	2026-05-16 17:51:05	
AccountID	AccountName	LastModifiedDate	
1	Trigger Test	2026-05-16 18:15:33	

Explanation of Views, Stored Procedures and Triggers

Views were created to simplify data retrieval for the Windows Form Application. They allow users to view related information without needing to understand complex SQL joins.

Stored Procedures were used to perform CRUD operations such as inserting, updating, and deleting records. They improve security, reusability, and performance.

Triggers were created to automatically enforce business rules and maintain data integrity.

References

- W3Schools SQL Views – https://www.w3schools.com/sql/sql_view.asp
- W3Schools SQL Joins – https://www.w3schools.com/sql/sql_join.asp
- GeeksforGeeks SQL Triggers – <https://www.geeksforgeeks.org/sql/sql-trigger/>

Explanation of Views, Stored Procedures and Triggers

Views were created to simplify data retrieval for the Windows Form Application. They allow users to view related information without needing to understand complex SQL joins.

Stored Procedures were used to perform CRUD operations such as inserting, updating, and deleting records. They improve security, maintainability, reusability, and performance.

Triggers were implemented to automatically enforce business rules and maintain data integrity within the database system.

Indexes were created on commonly searched fields such as ClientID, AccountID, ServiceID, SubscriptionID, and IncidentDate to improve query performance and database efficiency.

Conclusion

This assessment successfully implemented the CyberSecure Cloud Solutions database using SQL Server. All required tables, views, stored procedures, and triggers were created and tested successfully. The database supports secure and efficient management of clients, accounts, subscriptions, billing, and cybersecurity incidents.

References

- W3Schools SQL Views – https://www.w3schools.com/sql/sql_view.asp
- W3Schools SQL Joins – https://www.w3schools.com/sql/sql_join.asp
- GeeksforGeeks SQL Triggers – <https://www.geeksforgeeks.org/sql/sql-trigger/>